

CAHIER DES CHARGES

Projet FLOWLY

Application de Productivité Gamifiée
Desktop & Mobile Synchronisée

Projet Flowly App	Chef de projet Clément Hamimi
Version du document 1.0	Date Mars 2026

Document confidentiel

Table des Matières

1. Introduction et Contexte	3
2. Présentation du Projet	5
3. Périmètre et Exigences Fonctionnelles	8
4. Expérience Utilisateur (UX/UI)	16
5. Technologies et Hébergement	19
6. Sécurité et Conformité	30
7. Critères de Sélection des Prestataires	33
8. Méthodologie et Planning	35
9. Annexes	38

1. Introduction et Contexte

1.1 Objet du Document

Le présent cahier des charges a pour objet de définir les spécifications fonctionnelles, techniques et organisationnelles pour le développement de l'application Flowly, une solution de productivité gamifiée multi-plateforme (desktop et mobile).

1.2 Contexte du Projet

Dans un monde hyperconnecté où les distractions numériques sont omniprésentes, maintenir sa concentration est devenu un défi majeur. Les applications de productivité traditionnelles présentent plusieurs limites : manque d'engagement, approche punitive, absence de synchronisation entre appareils.

1.3 Objectifs du Projet

1.3.1 Objectifs Stratégiques

1. Créer une application de référence sur le marché de la productivité gamifiée
2. Atteindre 50 000 utilisateurs la première année
3. Obtenir un taux de conversion freemium → premium de 5%

1.3.2 Objectifs Opérationnels

4. Développer une application desktop (Windows, macOS) et mobile (iOS, Android)
5. Assurer une synchronisation temps réel entre tous les appareils
6. Intégrer un monde 3D personnalisable et immersif
7. Mettre en place un système de gamification complet

2. Présentation du Projet

2.1 Description Générale

Flowly est une application desktop et mobile synchronisée qui transforme la productivité en expérience ludique. L'utilisateur gagne des points en travaillant de manière concentrée, qu'il peut dépenser dans une boutique virtuelle pour personnaliser un monde 3D et débloquer des avantages.

□ Proposition de Valeur

Flowly est la seule application combinant tracking intelligent, blocage de distractions, gamification avancée, monde 3D personnalisable ET dimension sociale dans une solution synchronisée desktop/mobile.

2.2 Cibles Utilisateurs

- Étudiants (18-25 ans) : révisions, concentration, motivation par le jeu
- Professionnels tech (25-40 ans) : deep work, gestion des distractions
- Freelances & entrepreneurs : auto-discipline, suivi de productivité

2.3 Fonctionnalités Clés

- Timer Pomodoro avec détection automatique de distraction
- Blocage synchronisé d'applications et notifications (desktop + mobile)
- Système de points, niveaux, achievements et streaks
- Monde 3D personnalisable avec boutique virtuelle
- Sessions de focus en groupe et classements
- Analytics de productivité et coach IA
- Intégration calendriers (Google, Outlook)

3. Périmètre et Exigences Fonctionnelles

3.1 Périmètre du Projet

3.1.1 Dans le Périmètre

- Application Desktop pour Windows 10+ et macOS 11+
- Application Mobile pour iOS 15+ et Android 10+
- Synchronisation temps réel entre desktop et mobile
- Monde 3D personnalisable (Three.js)
- Système de gamification complet
- Intégrations calendrier (Google Calendar, Outlook)

3.1.2 Hors Périmètre

- Application Web (navigateur)
- Intégration smartwatch
- Réalité virtuelle / augmentée
- Mode hors-ligne complet

3.2 Modules Fonctionnels

N°	Module	Description
M1	Sessions Focus	Timer Pomodoro, tracking d'activité, détection distractions
M2	Gamification	Points, niveaux, achievements, streaks, quêtes
M3	Boutique & Monde 3D	Achat objets, personnalisation monde, avatar
M4	Bloqueur	Blocage apps/sites, notifications, sync multi-appareil
M5	Analytics	Statistiques, heatmap, IA Coach, objectifs
M6	Social	Profils, classements, groupes, sessions collectives
M7	Authentification	Inscription, connexion, OAuth, gestion compte
M8	Audio	Musique intégrée, sons ambiants, alertes sonores

3.3 Exigences Fonctionnelles Détaillées

3.3.1 M1 — Sessions Focus

Réf.	Fonctionnalité	Critères d'Acceptance	Priorité
F1.01	Timer Pomodoro configurable	Durée 5-120 min, pause courte/longue, son de fin	Haute
F1.02	Lancement synchronisé	Session active sur desktop + mobile en < 3s	Haute
F1.03	Tracking clavier/souris	Détection inactivité > 2 min, pause auto	Haute
F1.04	Tracking applications	Liste apps utilisées, catégorisation auto	Moyenne

F1.05	Détection distractions	Popup si réseau social ouvert > 10s	Haute
-------	------------------------	-------------------------------------	-------

3.3.2 M2 — Gamification

Réf.	Fonctionnalité	Critères d'Acceptance	Priorité
F2.01	Système de points	1 pt/min focus, bonus streak, malus distraction	Haute
F2.02	Niveaux et XP	100 niveaux, XP croissant logarithmique	Haute
F2.03	Achievements	30+ badges, notification à l'obtention	Moyenne
F2.04	Streaks	Compteur jours consécutifs, bonus progressif	Haute
F2.05	Quêtes quotidiennes	3 quêtes/jour, difficulté variable	Moyenne

3.3.3 M3 — Boutique et Monde 3D

Réf.	Fonctionnalité	Critères d'Acceptance	Priorité
F3.01	Boutique virtuelle	50+ objets au lancement, catégories, preview	Haute
F3.02	Monde 3D personnel	Rendu 60 FPS, sauvegarde auto	Haute
F3.03	Placement objets	Drag & drop, rotation, grille alignement	Haute
F3.04	Avatar personnalisable	20+ options, déblocage par niveau	Moyenne
F3.05	Visite monde autres	Mode lecture seule, chargement < 5s	Moyenne

3.3.4 M4 — Bloqueur de Distractions

Réf.	Fonctionnalité	Critères d'Acceptance	Priorité
F4.01	Blocage réseaux sociaux	Liste prédéfinie + personnalisable	Haute
F4.02	Blocage notifications	Désactivation système, whitelist critiques	Haute
F4.03	Sync desktop-mobile	Blocage simultané, délai < 3s	Haute
F4.04	Mode strict	Impossible désactiver avant fin timer	Moyenne

3.3.5 Autres Modules

- M5 Analytics : Dashboard stats, heatmap, IA Coach, intégrations calendrier
- M6 Social : Profils, classements, sessions groupe, organisations
- M7 Auth : Email/password, OAuth (Google, GitHub, LinkedIn), SSO
- M8 Audio : Playlists lofi/ambient, sons ambiants mixables

4. Expérience Utilisateur (UX/UI)

4.1 Principes Directeurs

- Simplicité : Actions principales en moins de 3 clics
- Motivation positive : Approche encourageante, jamais culpabilisante
- Immersion : Monde 3D comme élément central
- Feedback immédiat : Retour visuel pour chaque action < 100ms

4.2 Charte Graphique

Usage	Couleur	Description
Primaire	#2E75B6	Bleu confiance — Actions principales
Gamification	#7B68EE	Violet — Récompenses, XP
Succès	#2E7D32	Vert — Validations, streaks
Alerte	#E65100	Orange — Distractions

4.3 Parcours Utilisateur Clés

Onboarding

8. Écran de bienvenue
9. Connexion (email, Google, GitHub)
10. Tutoriel interactif (< 3 min, skippable)
11. Première session guidée avec récompense
12. Découverte du monde 3D

Session Focus Quotidienne

13. Dashboard avec suggestion session
14. Sélection durée et activation blocage
15. Timer avec ambiance sonore
16. Fin : récapitulatif, points, célébration

4.4 Accessibilité

- WCAG 2.1 niveau AA
- Contraste minimum 4.5:1
- Navigation clavier complète
- Support lecteurs d'écran

5. Technologies et Hébergement

Cette section présente les choix technologiques retenus suite aux veilles technologiques réalisées pour chaque composant du système.

5.1 Application Desktop (Frontend)

❑

Choix Retenu : Electron + React + TypeScript

Electron reste la solution la plus adaptée au contexte du projet, enrichie par des modules natifs pour les fonctionnalités système avancées. Cette stack offre un excellent compromis entre performances, ergonomie et maintenabilité.

5.1.1 Justification du Choix

Après analyse comparative (Tauri, Flutter Desktop, Qt, .NET MAUI), Electron a été retenu pour les raisons suivantes :

- Large écosystème et forte communauté
- Intégration facile avec React
- Support natif des notifications système
- Compatibilité avec Three.js pour le rendu 3D
- Rapidité de développement et partage de code

5.1.2 Limitation Identifiée

Electron seul ne suffit pas pour les interactions système avancées. Une architecture hybride est nécessaire :

- Electron gère l'interface utilisateur
- Des modules natifs (C++, Rust, Swift) gèrent les interactions système
- Communication via IPC (Inter-Process Communication)

5.1.3 Alternatives Écartées

Technologie	Raison de l'Écart
Tauri	Léger et performant, mais APIs systèmes encore limitées pour besoins avancés
Flutter Desktop	UI performante, mais intégration 3D web et OS plus complexe
Qt	Très puissant, mais lourd et coûteux en temps d'apprentissage
.NET MAUI	Peu adapté à la 3D et à la gamification

5.1.4 Stack Technique Desktop

Composant	Technologie	Usage
Runtime	Electron	Application desktop multi-plateforme
Framework UI	React	Composants réutilisables, hooks
Langage	TypeScript	Typage statique, maintenabilité

Moteur 3D	Three.js	Rendu du monde 3D
Intégration React 3D	React Three Fiber	Intégration naturelle avec React
Animations UI	Framer Motion	Animations d'interface fluides
Composants UI	Radix UI	Composants headless, accessibles
State Management	Zustand	Gestion état global légère
Communication	REST + WebSocket	API classique + temps réel
IPC	Electron IPC	Interactions système

5.1.5 Architecture Frontend

```
src/
├─ app/                (bootstrap, routing)
├─ features/           (focus, shop, world, social)
├─ shared/             (UI, hooks, utils)
├─ services/           (API, WebSocket, IPC)
├─ stores/             (state global)
└─ assets/             (images, fonts, 3D)
```

5.1.6 Performance et Optimisation

- Chargement différé des fonctionnalités lourdes (monde 3D)
- Optimisation des assets graphiques
- Limitation du rendu 3D hors interaction
- Utilisation des Web Workers pour calculs intensifs

5.2 Applications Mobiles

❑ Décision Architecturale Majeure : Natif Distinct

Contrairement à l'approche cross-platform habituelle, le projet Flowly nécessite un développement NATIF DISTINCT pour iOS et Android. Cette décision est motivée par une contrainte technique bloquante.

5.2.1 Pourquoi pas React Native ?

Après analyse, React Native ne peut PAS être utilisé pour ce projet :

Limitation	Détail
Screen Time API (iOS)	IMPOSSIBLE avec RN. Nécessite Swift natif + entitlement Apple spécifique (FamilyControls, ManagedSettings, DeviceActivity)
UsageStatsManager (Android)	Nécessite code Kotlin natif, intégration RN complexe via module custom lourd
Performance animations	Pont JS/natif introduit latence sur animations complexes de la DA gamifiée
Rendu custom DA	UI gamifiée personnalisée plus difficile à optimiser en RN

5.2.2 Stack iOS — SwiftUI Natif

iOS nécessite SwiftUI natif pour accéder aux APIs Screen Time d'Apple.

Composant	Technologie	Usage
Framework UI	SwiftUI	UI déclarative moderne Apple
Langage	Swift 5.9+	Langage officiel Apple
Blocage apps	FamilyControls	Screen Time API officielle Apple
Suivi activité	DeviceActivity	Monitoring usage apps iOS
Notifications	UserNotifications	UNUserNotificationCenter natif
Background	BackgroundTasks	BGAppRefreshTask pour sync
Auth OAuth	AuthenticationServices	Sign in with Apple, Google
Animations	Lottie iOS	Animations gamification
Charts	Swift Charts	Graphiques natifs iOS 16+
Monde 3D	WKWebView + Three.js	Bridge JS pour monde 3D
Stockage sécurisé	Keychain	Tokens, données sensibles
Stockage local	SwiftData	Persistance données
State Management	@Observable	iOS 17+ pattern natif
Architecture	MVVM	Model-View-ViewModel

5.2.3 Structure Projet iOS

```
Flowly/
├─ Core/          # modèles, services, networking
├─ Features/      # un dossier par feature
│   └─ Focus/
│       ├── FocusView.swift
│       ├── FocusViewModel.swift
│       └─ FocusModel.swift
├─ Shared/        # composants réutilisables
└─ Resources/     # assets, localization
```

5.2.4 Stack Android — Flutter

Flutter est la meilleure alternative cross-platform pour Android grâce au moteur de rendu Skia.

Composant	Technologie	Usage
Framework	Flutter 3.x	Moteur Skia performant
Langage	Dart 3.x	Null-safety, async natif
Monitoring apps	UsageStatsManager	Via Platform Channels (Kotlin)
Background tasks	WorkManager	Tâches différées
Foreground Service	Android Service	Session active arrière-plan
Notifications	NotificationManager	NotificationChannel API 26+
Animations	Rive / Lottie Flutter	Animations performantes
Charts	fl_chart	Graphiques Flutter
Monde 3D	WebView Flutter	Affichage monde Three.js
Stockage sécurisé	flutter_secure_storage	EncryptedSharedPreferences
Stockage local	Hive / Drift	NoSQL / SQLite typé
State Management	Riverpod	Moderne, typé, testable
HTTP Client	Dio	Client HTTP Flutter
Architecture	MVVM	Cohérence avec iOS

5.2.5 Structure Projet Flutter

```
lib/
├── core/           # network, storage, theme
├── features/       # feature-first
│   ├── focus/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   └── shared/     # widgets communs
```

5.2.6 Cohérence UX iOS / Android

Le monde 3D, le leaderboard, la boutique et les stats sont affichés via WebView commune (Three.js), assurant la cohérence visuelle. Pour les écrans natifs, un Design System partagé (couleurs, typographie, espacements) est défini et implémenté séparément.

5.2.7 Gestion État Global Mobile

- iOS (SwiftUI) : @Observable (iOS 17+) ou @StateObject + @EnvironmentObject
- Android (Flutter) : Riverpod — moderne, bien typé, compatible MVVM

5.3 Backend

❑ Choix Retenu : Node.js + NestJS

Node.js est le choix optimal pour ce projet. NestJS est le framework Node.js de référence en 2025 pour les applications d'entreprise (60k+ stars GitHub).

5.3.1 Justification Node.js

- Unification du langage JavaScript/TypeScript entre front et back
- Excellent support WebSocket pour la synchronisation desktop/mobile temps réel
- Modèle événementiel asynchrone parfait pour les applications temps réel
- Écosystème npm mature pour OAuth, WebSockets, intégration Three.js
- Scalabilité horizontale native avec clustering

5.3.2 Justification NestJS

- Architecture modulaire pour organiser les domaines (gamification, analytics, social)
- TypeScript natif réduisant les bugs et améliorant la maintenabilité
- Support intégré WebSockets, GraphQL, validation des données
- Injection de dépendances facilitant les tests
- Documentation exemplaire et communauté active
- Utilisé par Adidas, Roche, Autodesk (adoption enterprise)

5.3.3 API REST vs GraphQL

REST API est recommandé pour ce projet :

- Cache HTTP natif et efficace pour classements et statistiques
- Simplicité d'implémentation réduisant la complexité initiale
- Rate limiting par endpoint plus simple à configurer
- Meilleur support pour la synchronisation mobile/desktop
- NestJS permet d'ajouter GraphQL plus tard si nécessaire

5.3.4 Stack Technique Backend

Composant	Technologie	Usage
Runtime	Node.js 20 LTS	Support long terme, performances
Framework	NestJS 10+	Architecture modulaire, DI
Langage	TypeScript 5+	Typage statique end-to-end
API Style	REST	Cache HTTP, simplicité
Base de données	PostgreSQL 15+	ACID, relations, JSONB
ORM	Prisma	TypeScript natif, migrations auto
Cache	Redis	Sessions, classements, pub/sub
WebSocket	Socket.io	Sync temps réel
Queue	BullMQ	Tâches async (emails, analytics)

Validation	class-validator	DTOs validés NestJS
Documentation	OpenAPI/Swagger	Génération auto
Tests	Jest + Supertest	Unitaires + intégration

5.3.5 Architecture Monolithe Modulaire

L'architecture monolithe modulaire est le meilleur compromis pour ce projet :

- Complexité initiale faible, démarrage rapide
- Coût infrastructure réduit
- Pas de latence réseau entre services
- Déploiement simple
- Microservices pertinents seulement pour équipes > 10 développeurs
- Évolution possible vers microservices si la croissance l'exige

```
src/
├── modules/
│   ├── auth/           # JWT, OAuth, guards
│   ├── users/          # Profils, préférences
│   ├── focus/          # Sessions, tracking
│   ├── gamification/   # Points, niveaux, achievements
│   ├── shop/           # Boutique, achats
│   ├── world/          # Monde 3D, objets
│   ├── analytics/      # Stats, IA Coach
│   └── social/         # Groupes, classements
├── common/             # Filters, interceptors
└── prisma/             # Schema, migrations
```

5.3.6 Authentification JWT + OAuth 2.0

La contrainte multi-plateforme (desktop + mobile) oriente vers JWT :

Élément	Configuration
Access Token	JWT, durée 15-30 minutes
Refresh Token	JWT, durée 7-30 jours, rotation à chaque usage
Stockage Web	HttpOnly Secure Cookie (protection XSS)
Stockage Mobile	Keychain (iOS) / EncryptedSharedPreferences (Android)
Révocation	Blacklist Redis pour invalidation immédiate
Providers OAuth	Google, GitHub, LinkedIn via OAuth 2.0 + OpenID Connect

5.3.7 Gestion des Rôles (RBAC)

Rôle	Permissions
User	Accès standard : sessions, boutique, profil, stats personnelles
Premium	+ Fonctionnalités premium : IA Coach, analytics avancés
GroupAdmin	+ Gestion groupe/organisation : membres, stats collectives
Admin	Administration globale de la plateforme

Implémentation : NestJS Guards + CASL.js pour permissions fines.

5.3.8 Base de Données

PostgreSQL comme base principale pour les raisons suivantes :

- ACID compliance pour les transactions (points, achats)
- Relations complexes (utilisateurs, groupes, organisations, amis)
- Support JSONB natif pour données semi-structurées (configs monde 3D)
- Performance excellente pour requêtes complexes (classements, stats)
- Maturité (35+ ans), très stable

Redis pour le cache et temps réel :

- Cache des données fréquemment accédées
- Sessions JWT et tokens
- Classements temps réel
- File d'attente (BullMQ)
- Pub/sub pour WebSocket multi-instances

5.4 Infrastructure et DevOps

5.4.1 Conteneurisation

Environnement	Solution	Usage
Développement	Docker	Environnement local reproductible
Production	Kubernetes	Orchestration, scaling, haute dispo

Docker et Kubernetes sont complémentaires : Docker pour le packaging, Kubernetes pour l'orchestration en production.

5.4.2 Pipeline CI/CD — GitLab CI

GitLab CI est retenu pour centraliser code et pipelines sur une même plateforme.

Stage	Actions	Outils
Build	Compilation, bundling, images Docker	npm, Docker, Gradle, Xcode
Test	Unitaires, intégration, E2E	Jest, Supertest, Playwright
Security	Scan code, dépendances, secrets	Trivy, SonarQube, GitLab SAST
Quality	Lint, coverage, dette technique	ESLint, SonarQube
Deploy	Déploiement par environnement	Terraform, Ansible, kubectl

5.4.3 Environnements

Environnement	Déclencheur	Usage
Development	Push sur feature/*	Tests développeurs
Staging	Merge sur develop	Tests QA, validation PO
Production	Merge sur main + tag	Utilisateurs finaux

5.4.4 Infrastructure as Code

- Terraform : provisioning infrastructure (VPC, EC2/ECS, RDS, S3)
- Ansible : playbooks de configuration et déploiement

La reproductibilité des environnements est assurée par ces outils.

5.4.5 Gestion des Secrets

- Développement : fichiers .env (dans .gitignore)
- CI/CD : GitLab CI Variables (protected, masked)
- Production : GitLab Secrets ou service cloud public
- Règle absolue : aucun secret hardcodé dans le code

5.4.6 Monitoring et Observabilité

Aspect	Outil	Usage
Métriques	Prometheus	Collecte métriques système et applicatives

Dashboards	Grafana	Visualisation, alertes, tableaux de bord
Alerting	Grafana Alerts	Notifications Slack, email en cas d'anomalie

Prometheus + Grafana permettent d'anticiper les pannes et assurer une bonne disponibilité du service grâce au système d'alerting intégré.

5.4.7 Approche DevSecOps

L'approche DevSecOps est intégrée DÈS LE DÉBUT du projet pour éviter de recoder la sécurité après coup :

- Scan de code statique (SAST) : SonarQube dans la pipeline
- Scan de dépendances : Trivy, npm audit
- Scan d'images Docker : Trivy
- Lint et qualité de code : ESLint
- Rapports générés à chaque build

5.4.8 Déploiement Mobile

Plateforme	Pipeline	Prérequis
iOS	Xcode → TestFlight → App Store	Apple Developer (99\$/an), entitlement FamilyControls
Android	Android Studio → Google Play	Compte Play (25\$ one-time)
Automatisation	Fastlane + GitHub Actions	Build, sign, upload automatisés

5.5 Récapitulatif Stack Technologique

Couche	Technologie	Version
Desktop	Electron + React + TS	Electron 28+, React 18+
iOS	SwiftUI + Swift	iOS 15+, Swift 5.9+
Android	Flutter + Dart	Flutter 3.x, Dart 3.x
3D Engine	Three.js + R3F	Three.js r160+
Backend	NestJS + Node.js	NestJS 10+, Node 20 LTS
Database	PostgreSQL + Redis	PostgreSQL 15+, Redis 7+
ORM	Prisma	Prisma 5+
WebSocket	Socket.io	Socket.io 4+
CI/CD	GitLab CI	Latest
IaC	Terraform + Ansible	Terraform 1.5+
Containers	Docker + Kubernetes	K8s 1.28+
Monitoring	Prometheus + Grafana	Latest

5.6 Exigences de Performance

Métrique	Objectif	Acceptable
Temps de réponse API (P95)	< 200 ms	< 500 ms
Synchronisation desktop-mobile	< 3 secondes	< 5 secondes
Rendu 3D (FPS)	60 FPS	30 FPS
Démarrage application	< 3 secondes	< 5 secondes
Disponibilité (uptime)	99.5%	99%

6. Sécurité et Conformité

6.1 Sécurité Applicative

Menace	Protection	Implémentation
SQL Injection	Requêtes paramétrées	ORM Prisma exclusivement
XSS	Sanitization, CSP headers	class-validator, Helmet.js
CSRF	Tokens CSRF, SameSite cookies	csrf middleware
Brute Force	Rate limiting, captcha	@nestjs/throttler, reCAPTCHA
JWT Compromise	Tokens courts, rotation	Redis blacklist

6.2 Chiffrement

- En transit : TLS 1.3 obligatoire pour toutes les communications
- Au repos : Chiffrement AES-256 via AWS RDS
- Mots de passe : Hash bcrypt (cost factor 12)
- Tokens mobiles : Keychain (iOS), EncryptedSharedPreferences (Android)

6.3 Conformité RGPD

Droit	Implémentation	Délai
Accès	Export JSON/CSV complet des données	< 48h
Rectification	Modification profil en self-service	Immédiat
Effacement	Suppression complète compte et données	< 72h
Portabilité	Export format standard	< 48h
Opposition	Opt-out analytics et marketing	Immédiat

7. Critères de Sélection des Prestataires

7.1 Compétences Requises

- Développement Electron / React / TypeScript (3+ ans)
- Développement mobile natif : SwiftUI (iOS) ET Flutter (Android)
- Backend Node.js / NestJS avec architecture modulaire
- Expérience Three.js / WebGL pour applications 3D
- DevOps : CI/CD GitLab, Docker, Kubernetes

7.2 Critères d'Évaluation

Critère	Pondération	Éléments évalués
Compétences techniques	30%	Stack maîtrisée, certifications
Références et portfolio	25%	Projets similaires réalisés
Proposition technique	20%	Compréhension besoins, solutions
Budget	15%	Adéquation qualité/prix
Méthodologie	10%	Process, communication

7.3 Documents à Fournir

17. Présentation de la société
18. CV des intervenants
19. Portfolio de réalisations similaires
20. Proposition technique détaillée
21. Planning prévisionnel
22. Devis détaillé par phase

8. Méthodologie et Planning

8.1 Méthodologie Agile Scrum

- Sprints de 2 semaines
- Daily stand-up quotidien (15 minutes)
- Sprint Planning en début de sprint (2-3h)
- Sprint Review et Rétrospective en fin de sprint (1h + 1h)
- Backlog Refinement hebdomadaire (1h)

8.2 Phases du Projet

Phase	Durée	Livrables
Phase 0 : Cadrage	2 semaines	Spécifications, maquettes HD, architecture
Phase 1 : MVP	12 semaines	Sessions focus, points, blocage, auth
Phase 2 : V1.0	8 semaines	Monde 3D, boutique, analytics, social basique
Phase 3 : V1.5	8 semaines	IA Coach, intégrations calendrier, co-working
Phase 4 : V2.0	10 semaines	Battle pass, événements, boutique réelle

8.3 Jalons Clés

Jalon	Date	Critère de validation
Kick-off	Avril 2026	Équipe constituée, environnements prêts
MVP Beta	Juillet 2026	Fonctionnalités core testables
V1.0 Release	Septembre 2026	Lancement public
V1.5	Novembre 2026	IA Coach et intégrations
V2.0	Février 2027	Version complète

8.4 Critères d'Acceptance du Projet

23. 100% des fonctionnalités priorité Haute implémentées
24. 80% des fonctionnalités priorité Moyenne implémentées
25. Couverture de tests : 70% unitaires, 80% intégration
26. Performance : 60 FPS 3D, < 3s sync, < 200ms API
27. 0 vulnérabilité critique
28. Documentation complète

9. Annexes

9.1 Glossaire

Terme	Définition
Pomodoro	Technique de gestion du temps : 25 min travail, 5 min pause
Gamification	Application de mécaniques de jeu à des contextes non ludiques
Streak	Série de jours consécutifs d'objectif atteint
MVVM	Model-View-ViewModel — Pattern architectural
JWT	JSON Web Token — Standard d'authentification
IPC	Inter-Process Communication — Communication entre processus
Screen Time API	API Apple pour contrôler le temps d'écran sur iOS

9.2 Documents Annexés

- Annexe A : Maquettes détaillées (Figma)
- Annexe B : Diagrammes d'architecture
- Annexe C : Modèle de données (ERD)
- Annexe D : Spécifications API (OpenAPI)
- Annexe E : Veille technologique Frontend
- Annexe F : Veille technologique Backend
- Annexe G : Veille technologique Mobile
- Annexe H : Veille technologique DevOps

9.3 Équipe Projet

Organigramme

Chef de Projet Clément Hamimi				
Tech Lead Theo Evon		Product Owner Salma Rahim		
Dév Mobile Imrane Sallak	Dév Full-Stack Wilfried D. N.	DevOps Rezak Khorsi	Dév Full-Stack Yvann Hubert	Dév Full-Stack Buket M. Ozkan

Responsabilités

Rôle	Nom	Responsabilités
------	-----	-----------------

Chef de Projet	Clément Hamimi	Pilotage global, coordination, reporting
Tech Lead	Theo Evon	Architecture technique, revue de code
Product Owner	Salma Rahim	Vision produit, backlog, validation
Dév Mobile	Imrane Sallak	iOS (SwiftUI), Android (Flutter)
Dév Full-Stack	Wilfried Djoum Nguemtchue	Frontend Desktop, Backend NestJS
DevOps	Rezak Khorsi	CI/CD, infrastructure, monitoring
Dév Full-Stack	Yvann Hubert	Frontend Desktop, Backend NestJS
Dév Full-Stack	Buket Meltem Ozkan	Frontend Desktop, Backend NestJS

— *Fin du Cahier des Charges* —

FLOWLY — Cahier des Charges v1.0